Q1. (20 pts) How many lines does the following function print if it is called as `f(n,1)`? Write a recurrence relation. Give the base case clearly. Solve it by backward substitution. (You may assume that $n = 2^k$)

```
function f(n,m)
  if n <= m printline(«Take it easy»);
  else
       for i=1 to m
             return f(n/2,m*2)
```

$P(n,m) = m\,P(\frac{n}{2}, 2m)$

$P(2^k, 1) = 1.P(2^{k-1}, 2) = 2.P(2^{k-2}, 2^2) = 2.2^2.P(2^{k-3}, 2^3) = 2.2^2.2^3 P(2^{k-4}, 2^4)$

$= 2.2^2.2^3 \dots 2^{\frac{k}{2}-1} \underbrace{P(2^{\frac{k}{2}}, 2^{\frac{k}{2}})}_{1} = 2^{\frac{k}{2}(\frac{k}{2}-1)\cdot\frac{1}{2}} = 2^{\frac{k^2}{8}-\frac{k}{4}} = 2^{\frac{\log_2^2 n}{8}-\frac{\log_2 n}{4}}$

Q2. (20 pts) Write a recurrence relation for the worst case and best case time complexity of the following function. Write recurrence relations and solve by Master Theorem.

```
function f2(int A[0..n-1])
  if n < 10 SelectionSort(A[0..n-1])
  else {
       InsertionSort(A[0..⌈√n⌉-1])
       f2(A[..⌊3n/8⌋-1]);
       f2(A[⌈5n/8⌉..n-1]);
```

→ base.

$T_w(n) = 2T_w(\frac{3n}{8}) + n$ → worst case for the insertion sort. $\Theta((\sqrt{n})^2)$

$a=2 \quad b=\frac{8}{3} \quad d=1 \quad a<b^d \quad T_w(n) \in \Theta(n)$

$T_b(n) = 2T_b(\frac{3n}{8}) + \sqrt{n}$ → best case for the insertion sort.

$a=2 \quad b=\frac{8}{3} \quad d=\frac{1}{2} \quad a>b^d \quad T_b(n) \in \Theta(n^{\log_{\frac{8}{3}} 2})$

Q3. (28 pts) Given three integer arrays *A*, *B* and *C* of size *n*, *m* and *k* respectively (*n* > *m* > *k*). There are no duplicate elements in the arrays. The goal is to find a list $D = A \cap B \cap \neg C$. In other words the goal is to return a list that includes all the integers that exists both in *A* and *B*, but not in *C*.
(a) (13 pts) Design a brute-force algorithm. Give a pseudocode. Describe its time complexity.
(b) (15 pts) Design an algorithm with a better time complexity. Give step-by-step description of your algorithm and describe its time complexity. (Please do not give pseudocode!)

(a)

```
for i=1 to n
    for j=1 to m                    → This comparison is done n.m times in the worst case
        if A[i]=B[j] {
            0 ← inC
            for t=1 to k            → This comparison is done m.k times in the worst case
                if A[i]=C[k] 1 ← inC;
            if (inC=0) add A[i] to D.
```

Time complexity = $O(nm) + O(mk)$
= $O(nm)$

OR

```
for i=1 to n
    for j=1 to m
        if A[i]=B[j]                → n.m times in the worst case.
            add A[i] to D.

for p=1 to size(D)
    for q=1 to k                    → m.k times in the worst case.
        if D[p]=C[q]
            remove D[p] from D.
```

Time complexity = $O(nm)$.

(b) Sort B and C. $\Theta(m\log m) + \Theta(k\log k)$

Search all elements in A in B. using binary search. $O(n\log m)$

If exists, search in C using binary search. If not exists, add it to D. $O(n\log k)$

Time complexity: $O(n\log m)$.

Q4. (20 pts) Prove that order in which vertices are pushed onto a DFS stack (instead of the pop order) cannot be used to solve the topological sorting problem. (Hint: Give a small counter-example. You should show that neither push order nor its reverse is the solution.)



push order : a b c

But neither push order nor its inverse give a topological sort.

Q5. (22 pts) Given an array of positive and negative integers A[1], A[2], ..., A[n], describe a divide-and-conquer algorithm to find the contiguous subsequence with the largest sum (maximum sum subarray). Describe the time complexity of your algorithm. For example, if the array is {-5, 30, 40, -90, 25, 75, -5, -20, 90, -10}, the answer is {25, 75, -5, -20, 90}.

maximum sum subarray, MSS

Divide the array into two parts.

Find MSS in the left part. ⇒ subsequence L

Find MSS in the right part. ⇒ subsequence R.

Find the MSS ending at the middle element. ⇒ mid L

Find the MSS starting from the middle element ⇒ mid R

} Requires single iteration, add elements one by one.

append mid L & mid R ⇒ subsequence M.

return largest of subsequence L, subsequence R and subsequence M.

(you may just return the start index & end index)

Time complexity = $2T\left(\frac{n}{2}\right) + n$.

$\in O(n \log n)$

